



PYTHON LISTS

IT 5/L – ELECTIVE 2



○ Collection

A collection is a single variable used to store multiple values.

1. **List** (`[]`): Ordered and changeable. Duplicates are allowed.
2. **Set** (`{}`): Unordered and immutable. Adding and removing items is allowed, but duplicates are not allowed.
3. **Tuple** (`()`): Ordered and unchangeable. Duplicates are allowed, and tuples are even faster than lists.
4. **Dictionary** (`{}`): Unordered collections of items. Each item has a key-value pair, and they are mutable.



○ Python Lists

- Lists are used to store multiple items in a single variable.
- List items are ordered, changeable, and allow duplicate values.
- List items are indexed and you can access them by referring to the index number

Example:

```
list = [item1, item2]
```



○ Python Lists

Instead of using this:

```
fruit1 = "Cherry"
```

```
fruit2 = "Apple"
```

```
fruit3 = "Banana"
```

We can use:

```
fruits = ["Cherry", "Apple", "Banana"]
```



○ Example

```
tagum_brgy = ["Apokon", "Bincungan", "Busaon", "Canocotan",  
"Cuambugan", "La Filipina", "Liboganon", "Madaum", "Magdum", "Magugpo  
Poblacion", "Magugpo East", "Magugpo North", "Magugpo South", "Magugpo  
West", "Mankilam", "New Balamban", "Nueva Fuerza", "Pagsabangan",  
"Pandapan", "San Agustin", "San Isidro", "San Miguel", "Visayan Village"]  
print(tagum_brgy)
```

Output:

```
['Apokon', 'Bincungan', 'Busaon', 'Canocotan', 'Cuambugan', 'La Filipina',  
'Liboganon', 'Madaum', 'Magdum', 'Magugpo Poblacion', 'Magugpo East', 'Magugpo  
North', 'Magugpo South', 'Magugpo West', 'Mankilam', 'New Balamban', 'Nueva  
Fuerza', 'Pagsabangan', 'Pandapan', 'San Agustin', 'San Isidro', 'San Miguel',  
'Visayan Village']
```



○ List Indexing

```
tagum_brgy = ["Apokon", "Bincungan", "Busaon", "Canocotan",  
"Cuambugan", "La Filipina", "Liboganon", "Madaum", "Magdum", "Magugpo  
Poblacion", "Magugpo East", "Magugpo North", "Magugpo South", "Magugpo  
West", "Mankilam", "New Balamban", "Nueva Fuerza", "Pagsabangan",  
"Pandapan"]
```

```
print(tagum_brgy[2])
```

Output:
Busaon



○ Negative Indexing

```
tagum_brgy = ["Apokon", "Bincungan", "Busaon", "Canocotan",  
"Cuambugan", "La Filipina", "Liboganon", "Madaum", "Magdum", "Magugpo  
Poblacion", "Magugpo East", "Magugpo North", "Magugpo South", "Magugpo  
West", "Mankilam", "New Balamban", "Nueva Fuerza", "Pagsabangan",  
"Pandapan"]
```

```
print(tagum_brgy[-2])
```

Output:
Pagsabanga
n



○ Range of Indexes

```
tagum_brgy = ["Apokon", "Bincungan", "Busaon", "Canocotan",  
"Cuambugan", "La Filipina", "Liboganon", "Madaum", "Magdum", "Magugpo  
Poblacion", "Magugpo East", "Magugpo North", "Magugpo South", "Magugpo  
West", "Mankilam", "New Balamban", "Nueva Fuerza", "Pagsabangan",  
"Pandapan"]
```

```
print(tagum_brgy[3:6])
```

Output:

```
['Canocotan', 'Cuambugan', 'La Filipina']
```



○ Append Items

- To add an item to the end of the list, use the `append()` method:

Example:

- Using the `append()` method to append an item:

```
thislist = ["apple", "banana", "cherry"]  
thislist.append("orange")  
print(thislist)
```

Output:

```
['apple', 'banana', 'cherry', 'orange']
```





Insert Items

- To insert a list item at a specified index, use the `insert()` method.
- The `insert()` method inserts an item at the specified index:

Example

- Insert an item as the second position:

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(1, "orange")  
print(thislist)
```

Output:

```
['apple', 'orange', 'banana', 'cherry']
```



○ Extend Items

- To append elements from another list to the current list, use the `extend()` method.

Example

- Add the elements of tropical to thislist:

```
thislist = ["apple", "banana", "cherry"]  
tropical = ["mango", "pineapple", "papaya"]  
thislist.extend(tropical)  
print(thislist)
```

Output:

```
['apple', 'banana', 'cherry', 'mango', 'pineapple', 'papaya']
```



○ Remove Items

Remove Specified Item

- The `remove()` method removes the specified item.

Example

Remove "banana":

```
thislist = ["apple", "banana", "cherry"]  
thislist.remove("banana")  
print(thislist)
```

Output:

```
['apple', 'cherry']
```



○ Remove Specified Index

The `pop()` method removes the specified index.

Example

Remove the second item:

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop(1)  
print(thislist)
```

Note: If you do not specify the **index**, the `pop()` method removes the **last item**.

Output:

```
['apple', 'cherry']
```



○ Remove the last item

Example

➤ Remove the last item:

```
thislist = ["apple", "banana", "cherry"]  
thislist.pop()  
print(thislist)
```



○ Remove list using **del** keyword

➤ The del keyword also removes the specified index:

Example

➤ Remove the first item:

```
thislist = ["apple", "banana", "cherry"]  
del thislist[0]  
print(thislist)
```



- The **del** keyword can also delete the list completely

Example

➤ Delete the entire list:

```
thislist = ["apple", "banana", "cherry"]  
del thislist
```



○ Clear List

- The `clear()` method empties the list.
- The list still remains, but it has no content.

Example

Clear the list content:

```
thislist = ["apple", "banana", "cherry"]  
thislist.clear()  
print(thislist)
```



○ Copy Lists

- You cannot copy a list simply by typing `list2 = list1`, because: `list2` will only be a reference to `list1`, and changes made in `list1` will automatically also be made in `list2`.
- There are ways to make a copy, one way is to use the built-in List method `copy()`.

Example

Make a copy of a list with the `copy()` method:

```
thislist = ["apple", "banana", "cherry"]  
mylist = thislist.copy()  
print(mylist)
```





➤ Another way to make a copy is to use the built-in method `list()`.

Example

Make a copy of a list with the `list()` method:

```
thislist = ["apple", "banana", "cherry"]  
mylist = list(thislist)  
print(mylist)
```



○ Nested List

```
vegetables = ["carrot", "potato", "spinach"]
```

```
fruits = ["apple", "banana", "orange"]
```

```
foods = [vegetables, fruits]
```

```
print(foods)
```

Output:

```
[['carrot', 'potato', 'spinach'], ['apple', 'banana', 'orange']]
```



- More on lists

<https://docs.python.org/3/tutorial/datastructures.html>





QUIZ TIME



○ QUIZ #1-4

Given the following list:

```
fruits = ["Strawberries", "Nectarines", "Apples", "Grapes",  
"Peaches", "Cherries", "Pears"]
```

What will be the output of the following code?

1. `fruits[3]`
2. `fruits[4]`
3. `fruits[-5]`
4. `fruits[-4]`



○ QUIZ #5-7

Given the code below:

```
fruits = ["Strawberries", "Nectarines", "Apples", "Grapes",  
"Peaches", "Cherries", "Pears"]
```

```
fruits[-1] = "Melons"
```

```
fruits.append("Lemons")
```

```
print(fruits)
```

What do you think will be printed?



○ QUIZ #8-10

Given the code below:

```
fruits = ["Strawberries", "Nectarines", "Apples", "Grapes", "Peaches",  
"Cherries", "Pears"]  
vegetables = ["Spinach", "Kale", "Tomatoes", "Celery", "Potatoes"]
```

```
dirty_dozen = [fruits, vegetables]  
print(dirty_dozen[1][1])
```

What will be printed?





R A N D O M M O D U L E



○ Randomization

Python Random module is an in-built module of Python that is used to generate random numbers in Python. These are pseudo-random numbers means they are not truly random. This module can be used to perform random actions such as generating random numbers, printing random a value for a list or string, etc.



○ Randint

- Returns a random integer between a and b (both inclusive). This also raises a `ValueError` if $a > b$.

Example:

```
import random
random_integer = random.randint(1,10)
print(random_integer)
```



○ More Python Random Modules

<https://www.askpython.com/python-modules/python-random-module-generate-random-numbers-sequences>

